

Ella Energy Full-Stack Engineering Case

Belgian Weather Explorer

Thanks for making it to this stage. This case mirrors the kind of work you'd do at Ella: ingesting external time-series data reliably, exposing it through an API, and making it useful in a dashboard. We care more about your judgment and reasoning than about polish.

Timebox: aim for ~4 hours. The full scope is deliberately more than 4 hours of work. Cut scope where you must, but cut it *deliberately*: document what you skipped and why in your README. How you prioritize under constraint is part of what we evaluate.

The task

Build a small system that ingests Belgian weather data from the [Open-Meteo API](#) into PostgreSQL, serves it through a Python API, and displays it in a web dashboard.

1. Data ingestion into PostgreSQL

Ingest **forecast data** (not just historical observations) for at least three Belgian cities.

Note that forecasts get revised: Open-Meteo will return different values for the same city and target hour depending on when you ask. Your data model should handle this. We want to be able to answer both *"what is the latest forecast for Ghent tomorrow at 14:00?"* and *"what did the forecast for that hour look like yesterday?"*

Your ingestion pipeline must be safe to operate:

- **Idempotent**: running the ingest twice in a row must not create duplicates or corrupt data.
- **Atomic**: if a run fails halfway (API timeout, bad response, crash), the database must not be left with partially written data.
- **Resilient**: Open-Meteo being down or returning garbage should be handled gracefully. Log it and skip it; don't crash the pipeline or poison the dataset.

2. API with FastAPI (Python)

Build a FastAPI service that reads from the database. Minimum endpoints:

- Latest forecast for a city
- Forecast history for a given city and target time (how the forecast evolved)

Design the rest of the API surface as you see fit. We're interested in your choices around response shapes, query parameters, and error handling.

3. Dashboard with Next.js and shadcn/ui

A simple dashboard consuming your API. One or two well-executed views beat five rushed ones. At minimum: pick a city, see its forecast. If you want a stretch goal, visualize how a forecast changed across ingestion runs.

Bonus (optional)

- The ingest runs automatically on a schedule (e.g. daily). A cron container in your compose setup or a GitHub Action are both fine; tell us why you picked what you picked.

Ground rules

- **Use Claude Code** (or a comparable AI coding agent) and **commit your spec/plan files as part of the repo**. We want to see how you direct the tool: how you decompose the problem, what you specify upfront, and what you correct afterwards. If you don't have access to Claude Code, let us know and we'll sort it out.
- **Everything runs via docker compose up** from a fresh clone. No manual setup steps beyond what's in the README. We will test this literally.
- **Explain your architectural choices** in the README: data model (especially how you handle forecast revisions), failure handling, and anything you'd do differently with more time.

Deliverables

1. A Git repository (public or shared with us privately) containing:
 - Source code, spec files, and `docker-compose.yml`
 - A README covering: how to run it, your architecture decisions, and what you cut for time

2. That's it. No slides, no video.

What happens next

We'll review your submission, then schedule a ~45-minute technical debrief. Expect us to ask you to reason about extensions live: adding a second data source, scaling to thousands of locations, that kind of thing. You should understand every part of your submission well enough to modify it on the spot.

What we evaluate

- **Data modeling**, especially the forecast-revision problem
- **Operational safety**: idempotency, atomicity, failure handling
- **API design**: sensible, consistent, honest about errors
- **Prioritization**: what you cut and how you justified it
- **AI-assisted workflow**: the quality of your specs and how well you understand the result

Questions before or during the case are welcome. Asking good questions is signal, not weakness.

Good luck, and have fun with it.

The Ella Energy team